

ЛЕКЦИЯ 7. ДИНАМИЧЕСКОЕ ПРОГРАММИРОВАНИЕ. ПРИНЦИП ОПТИМАЛЬНОСТИ БЕЛЛМАНА

Содержание:

1. Понятие многошагового процесса. Постановка задачи оптимального управления многошаговым процессом. Принцип оптимальности Р.Беллмана.
2. Метод динамического программирования, основное функциональное соотношение метода. Прямая и обратная прогонка.
3. Программное управление и управление с обратной связью (синтез управления).

Метод динамического программирования – один из наиболее известных математических методов современной теории управления, предложен в конце 50-х годов американским математиком Ричардом Беллманом. Метод быстро получил широкое распространение, чему способствовали ярко и доходчиво написанные книги самого Беллмана, они были быстро переведены на русский язык и изданы у нас в стране [1-3]. Вскоре стало ясно, что метод динамического программирования тесно связан с классическим методом Гамильтона-Якоби в аналитической механике (для систем с непрерывным временем) и с последовательным анализом Вальда (для систем с дискретным временем). Однако весьма общая и отчетливая формулировка метода динамического программирования, данная Беллманом, а также многочисленные приложения метода к разнообразным проблемам теории принятия решений, экономики, биологии и других областей знания способствовали закреплению этого метода как одного из важнейших инструментов теории управляемых процессов.

1. Многошаговый процесс. Постановка задачи оптимального управления многошаговым процессом.

Рассматривается управляемая система. Предполагается, что время t изменяется дискретно и принимает целочисленные значения $0, 1, \dots, T$. Так, для процессов в экономике и экологии, дискретным значениям времени могут отвечать дни, месяцы или годы, а для процессов в электронных устройствах интервалы между соседними моментами, равные времени срабатывания устройства.

Состояние системы в каждый момент времени характеризуется n -мерным вектором x с компонентами $x_1, x_2, \dots, x_n$.

Предполагается, что на каждом шаге на систему оказывается **управляющее воздействие** при помощи m -мерного вектора управления u с компонентами $u_1, u_2, \dots, u_m$. На выбор управления накладываются ограничения, которые в достаточно общей форме можно представить в виде

$$u(t) \in U, t = 0, 1, \dots, T-1 \quad (1)$$

Здесь U – заданное множество в n -мерном пространстве.

Под влиянием выбранного в момент t управления система переходит в следующий момент времени $(t+1)$ в новое состояние. Этот переход можно описать соотношением

$$x(t+1) = f(x(t), u(t)), t = 0, 1, \dots, T-1, \quad (2)$$

где $f(x, u)$ – n -мерная функция от $(n+m)$ аргументов, она характеризует динамику рассматриваемой системы. Функция $f(x, u)$ предполагается известной, она отвечает принятой математической модели.

Зададим начальное состояние системы

$$x(0) = x_0, \quad (3)$$

где x_0 – заданный n -мерный вектор.

Итак, многошаговый процесс управления описывается соотношениями (1)-(3). Процедура расчета конкретного процесса сводится к следующему: если в некоторый

момент t состояние системы $x(t)$ известно, то для определения состояния $x(t+1)$ в следующий момент необходимо выполнить две операции:

- 1) выбрать допустимое управление $u(t)$, удовлетворяющее условию (1);
- 2) рассчитать состояние $x(t+1)$ в следующий момент времени, согласно (2)-(3).

Так как начальное состояние системы задано, то описанную процедуру можно пошагово выполнить для всех $t=0, 1, \dots, T-1$.

Последовательность состояний $x(0), x(1), \dots, x(T)$ называется траекторией системы.

Выбор управления на каждом шаге содержит определенный произвол, который исчезает, если задать **цель управления** в виде минимизации (или максимизации) некоторого **критерия оптимальности** (целевой функции). В результате возникает следующая **постановка задачи оптимального управления многошаговым процессом**.

Пусть задан некоторый критерий качества процесса управления (критерий оптимальности) вида

$$J(x, u) = F(x(T)) + \sum_{t=1}^{T-1} f^0(x(t), u(t)) \quad (4)$$

Здесь $f^0(x, u)$ и $F(x)$ - заданные скалярные функции своих аргументов, T - момент окончания процесса, $T > 0$. Функция f^0 отражает **расход средств или энергии на каждом шаге процесса**, а функция F - **характеризует оценку конечного состояния системы**.

Задача оптимального управления формулируется как задача определения управлений $u(0), u(1), \dots, u(T-1)$ из допустимого множества (1) и траектории $x(0), x(1), \dots, x(T)$, удовлетворяющей условиям (2)-(3), которые в совокупности доставляют минимальное значение критерию (4).

Оптимальное управление обычно обеспечивает **наименьшие затраты средств, ресурсов, энергии, наименьшее отклонение от заданной цели или заданной траектории процесса**.

Наряду с этим может рассматриваться и задача **максимизации** критерия вида (4), например, если речь идет о максимизации **дохода** или **объема производства**. Нетрудно видеть, что максимизация критерия J эквивалентна минимизации критерия $(-J)$. Поэтому простая замена знака у функций f^0 и F в (4) переводит задачу максимизации критерия в задачу минимизации. Далее всюду для определенности рассматриваем задачу о минимизации критерия (4).

Примеры управляемых многошаговых процессов

1. Проектирование ракеты. Предстоит спроектировать N -ступенчатую космическую ракету с заданным стартовым весом G . Известен также вес H космического корабля, который должен быть выведен на траекторию полета с помощью последней ступени ракеты-носителя. За время работы очередной ступени ракета получает добавочную скорость Δv , зависящую от веса P того груза, который несет эта ступень и от веса Q самой ступени: $\Delta v = f(P, Q)$. Требуется найти распределение веса между ступенями, при котором скорость корабля после отделения всех ступеней ракеты-носителя будет максимальной.

Обозначим через $u(t)$ **вес** t -й ступени, считая от корабля (рис. 1). Вес последней ступени, отделяемой после всех остальных ступеней и непосредственно выводящей корабль на траекторию полета, будет $u(1)$, вес предпоследней ступени $u(2)$ и т.д. Вес всего космического корабля вместе с примыкающими к нему t ступенями обозначим $x(t)$, $t=0, 1, 2, \dots, N$.

$$\text{Тогда} \quad x(t) = x(t-1) + u(t), \quad t=1, 2, \dots, N. \quad (2')$$

Задание веса корабля и стартового веса ракеты определяет граничные условия:

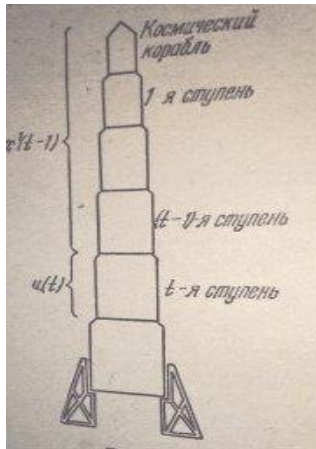
$$x(0) = H, \quad x(N) = G. \quad (3')$$

Добавочная скорость, сообщаемая за время работы t -й ступени, равна

$$\Delta v = f(x(t-1), u(t)),$$

так как $u(t)=Q(t)$ – вес t -й ступени, а $x(t-1)=P(t)$ – вес того груза, который она несет. Общая скорость корабля, сообщаемая ему всеми ступенями ракеты-носителя, равна

$$I(x,u) = \sum_{t=1}^N f(x(t-1), u(t)). \quad (4')$$



Задача о максимуме скорости космического корабля: для многошагового управляемого объекта найти такую траекторию $x(0), x(1), \dots, x(N)$ и управление $u(1), u(2), \dots, u(N)$, удовлетворяющие уравнению движения (2') с заданными граничными условиями $x(0)=H$, $x(N)=G$, для которых суммарная скорость $I(x,u)$ принимает наибольшее значение.

Рис.1. Условная схема для формализации уравнений движения в задаче распределения веса между ступенями космической ракеты.

2. Нелинейная транспортная задача. Число складов (источков) для простоты будем считать равным трем, число заводов-потребителей (стоков) равным N . Задача с правильным балансом, $\sum_{i=1,2,3} a_i = \sum_{j=1,2,\dots,N} b_j$. Стоимость перевозки u единиц сырья с i -го склада на j -й завод зависит от количества перевозимого сырья u . Обозначим эту нелинейную функцию стоимости через $c_{ij}(u)$. Задача состоит в том, чтобы перевезти сырье со складов на заводы-потребители с минимальными транспортными расходами.

Для составления плана перевозок, намечаем, прежде всего, количество сырья, которое надо завезти с первого и второго складов на 1-й завод. Затем количество сырья, перевозимого с этих же складов на 2-й завод, на 3-й завод и т.д., на N -й завод. Количество сырья, поставляемого третьим складом, определится тогда однозначно – на каждый завод надо довести с третьего склада недостающее до его потребностей количество сырья. Обозначим через $u_1(j)$ количество сырья, поставляемое на j -й завод первым складом, через $u_2(j)$ – вторым складом. Поскольку потребность j завода в сырье равна b_j , с третьего склада на j -й завод надо будет завезти сырье в количестве

$$b_j - u_1(j) - u_2(j).$$

Задача состоит в том, чтобы выбрать числа $u_1(j)$, $u_2(j)$, $j=1,2,\dots,N$. По смыслу задачи $u_1(j)$, $u_2(j)$ – неотрицательные, их сумма не должна превышать b_j :

$$U: \quad u_1(j) \geq 0, \quad u_2(j) \geq 0, \quad u_1(j) + u_2(j) \leq b_j, \quad j=1,2,\dots,N. \quad (1')$$

Иными словами, управление должно находиться в треугольнике, показанном на рис. 2.

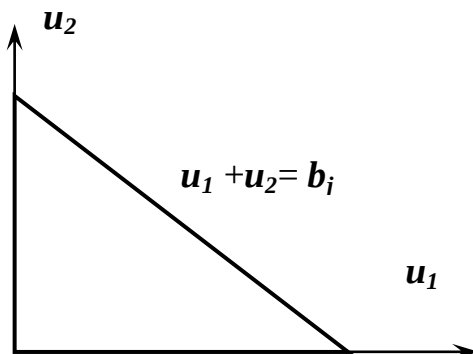


Рис. 2. Область допустимых управлений U в нелинейной транспортной задаче

Чтобы не превысить запасы сырья, имеющиеся на каждом складе, будем вести учет запланированного к вывозу сырья. Обозначим количество сырья, вывезенного с 1 склада на первые j заводов $x_1(j)$, а со второго склада $x_2(j)$. Очевидно, что

$$\begin{aligned} x_1(j) &= u_1(1) + u_1(2) + \dots + u_1(j), \\ x_2(j) &= u_2(1) + u_2(2) + \dots + u_2(j), \quad j=1, 2, \dots, N. \end{aligned}$$

При планировании поставок на j завод получим «уравнения движения» многошагового процесса:

$$x_1(j) = x_1(j-1) + u_1(j), \quad x_2(j) = x_2(j-1) + u_2(j), \quad j=1, 2, \dots, N. \quad (2'')$$

$$\text{с начальными условиями} \quad x_1(0)=0, \quad x_2(0)=0. \quad (3'')$$

Рассматривается задача с правильным балансом, запас равен спросу, в результате после завоза сырья на все N заводов склады остаются пустыми, $x_1(N)=a_1$, $x_2(N)=a_2$. Стоимость всех перевозок будет равна

$$I = \sum_{j=1}^N [c_{1j}(u_1(j)) + c_{2j}(u_2(j)) + c_{3j}(b_j - u_1(j) - u_2(j))]. \quad (4'')$$

Транспортная задача принимает следующую формулировку: для многошагового управляемого процесса (2'') найти *траекторию* $x_1(j)$, $x_2(j)$, $j=0, 1, \dots, N$ и *управление* $u_1(j)$, $u_2(j)$, $j=1, 2, \dots, N$, удовлетворяющие условиям (1')-(3''), при которых сумма (4'') принимает наименьшее возможное значение.

Принцип оптимальности Беллмана

Элементарный подход. При помощи соотношений (2) состояние системы в каждый последующий момент времени выражается через ее состояние и управление в предыдущий момент времени. Применяя это соотношение многократно, можно выразить состояния системы во все моменты времени только через начальное состояние $x(0)$ и управления в предшествующие моменты. В результате получим из (4)

$$J = f^0(x(0), u(0)) + f^0(f(x(0), u(0)), u(1)) + \dots = \Phi(x(0), u(0), u(1), \dots, u(T-1)).$$

Здесь Φ - некоторая громоздкая, но, вообще говоря, известная функция своих аргументов. В результате такого подхода поставленная задача оптимального управления сводится к задаче минимизации функции Φ по переменным $u(0), u(1), \dots, u(T-1)$, то есть всего T -м аргументов. При больших значениях параметра T решение задачи представляет затруднение даже для мощных современных компьютеров. Дополнительное осложнение вызвано тем, что переменные $u(t)$ должны удовлетворять ограничениям (1).

Принципиально иной подход к поставленной проблеме дает метод динамического программирования.

Сформулированный Р. Беллманом принцип оптимальности гласит: *отрезок оптимального процесса от любой его точки до конца процесса сам является оптимальным процессом с началом в данной точке.*¹

Принцип оптимальности легко доказывается от противного. Пусть $x(t) = x^*$ - некоторая точка оптимальной траектории в задаче (1)-(4), то есть состояние системы вдоль оптимального процесса в некоторый промежуточный момент t , $0 < t < T$.

Рассмотрим следующую задачу оптимального управления многошаговым процессом:

¹ Согласно книге Р.Беллмана «Динамическое программирование», Изд-во Иностранная литература, М., 1960, принцип оптимальности формулируется следующим образом: «Оптимальное поведение обладает тем свойством, что каковы бы ни были первоначальное состояние и решение в начальный момент, последующие решения должны составлять оптимальное поведение относительно состояния, получающегося в результате первого решения» (стр. 105).

$$\begin{aligned}
x(k+1) &= f(x(k), u(k)), x(t) = x^*, \\
u(k) &\subseteq U, \\
k &= t, t+1, \dots, T-1 \\
J_t &= F(x(T)) + \sum_{k=t}^{T-1} f^0(x(k), u(k)) \rightarrow \min
\end{aligned} \tag{5}$$

Рассуждая от противного, предположим, что отрезок этого процесса от момента t до момента T **не является оптимальным** для системы (5) в смысле критерия качества при начальном условии $x(t) = x^*$.

Значит, существуют допустимое управление $u(k)$ и соответствующая ему траектория $x(k)$, для которых критерий J_t из (5) принимает меньшее значение, чем на исходном процессе. На рис. 3 исходная оптимальная траектория $x(t)$ показана красной линией, а траектория $x(k)$ - синей.

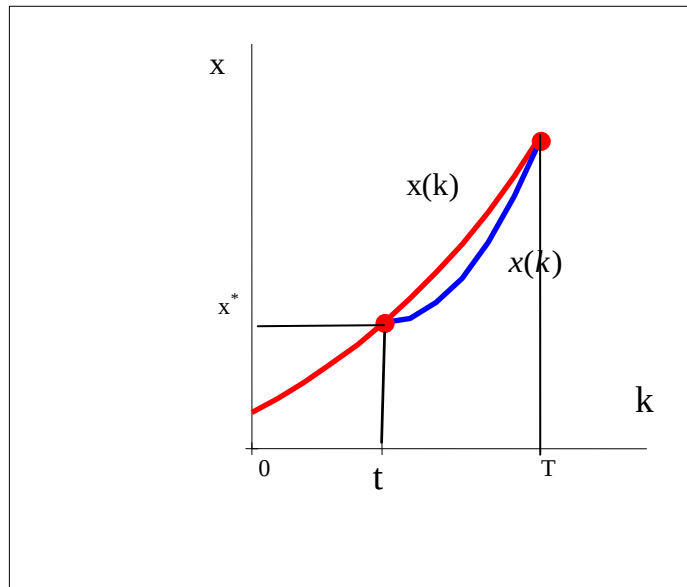


Рис. 3. Оптимальная исходная траектория $x(k)$ показана красной линией, «улучшенная» траектория $x(k)$ - синей

Наряду с исходным оптимальным процессом $x(k)$, $k = 0, 1, \dots, T$, рассмотрим процесс, состоящий из двух участков: исходного процесса $x(k)$ при $k = 0, 1, \dots, t$ и "улучшенного" процесса $x(k)$ при $k = t+1, \dots, T$. Для этого составного процесса критерий J из (4) будет иметь меньшее значение, чем для исходного процесса, так как сумма первых t слагаемых в (4) для составного процесса останется той же, что и для исходного процесса, а сумма остальных слагаемых, равная J_t , уменьшится по сравнению с исходным процессом. Данное утверждение означает, что исходный процесс не является оптимальным, а это противоречит условию, согласно которому $x(k)$, $k=0,1,\dots,T$ — *оптимальный процесс*.

Таким образом, принцип оптимальности доказан. Столь простое доказательство наводит на мысль о тривиальности этого принципа. Однако это не так: **принцип оптимальности является следствием аддитивности критерия оптимальности (4) и не**

имеет места в случае неаддитивного критерия, например для критерия, являющегося некоторой функцией от критериев вида (4).

2. Метод динамического программирования, основное функциональное соотношение метода

Обозначим: $S(t, x)$ – минимальное значение критерия качества J_t из (5) для оптимального процесса, начинающегося в момент t в точке $x(t) = x^*$.

$$S(t, x) = \min_{u(k) \subseteq U} J_t, \text{ то есть } S(t, x) = \min_{u(k) \subseteq U} [F(x(T)) + \sum_{k=t}^{T-1} f^0(x(k), u(k))].$$

Здесь минимум ищется по всем $u(k), k=t, \dots, T-1$.

Этот оптимальный процесс можно представить состоящим из двух участков:

первого шага с номером t , на котором выбирается управление $u(t)$ и
остальной части (от момента $t+1$ до конца процесса).

Операцию минимизации критерия качества J_t по u будем проводить, согласно **принципу сечений**. Зафиксируем $u(t)$ и будем минимизировать J_{t+1} по $u(k)$ для $k>t$. Результат минимизации будет функцией зафиксированного параметра, то есть $u(t)$. Итак,

$$\min_{u \subseteq U} J_t = \min_{u(t) \subseteq U_t} \min_{\substack{u(k) \subseteq U_k(u(t)), \\ k=t+1, \dots, T-1}} J_t = \min_{u(t) \subseteq U_t} \min_{\substack{u(k) \subseteq U_k(u(t)), \\ k=t+1, \dots, T-1}} [f^0(x(t), u(t)) + J_{t+1}(x(t+1), u(t+1))]$$

Здесь множество допустимых управлений $U_k(u(t))$ тоже зависит от зафиксированного управления $u(t)$.

Сумма в квадратных скобках – это J_t , то есть выделено первое слагаемое, которое заметим, не зависит от $u(k)$, начиная с $k=t+1, \dots$. Поэтому его можно вынести из-под знака внутреннего минимума.

Теперь $u(t)$ перестаем считать фиксированным, «отпускаем» и минимизируем по $u(t)$ результат минимизации J_{t+1} по $u(k)$ для $k>t$:

$$\min_{u \subseteq U} J_t = \min_{u(t) \subseteq U_t} [f^0(x(t), u(t)) + \min_{\substack{u(k) \subseteq U_k(u(t)), \\ k=t+1, \dots, T-1}} J_{t+1}(x(t+1), u(t+1))]$$

Таким образом, вклад в критерий качества первого участка процесса равен $f^0(x, u)$, а вклад второго участка можно, выразить через введенную выше функцию S в виде $S(t+1, x(t+1))$. Учитывая, что управление на первом участке должно выбираться из условия минимизации критерия J_t при ограничении (1), получим равенство

$$S(t, x(t)) = \min_{u(t) \subseteq U} [f^0(x(t), u(t)) + S(t+1, x(t+1))], \quad (6)$$

оно является **основным функциональным соотношением метода динамического программирования**.

Здесь и далее для определенности предполагаем, что функция S , как и ранее введенные в (2), (4) функции f, R, F , непрерывна. Следует заметить, что в (6) функция $S(t+1, x(t+1))$ *зависит от $u(t)$* . Подставляя в соотношение (6) равенство (2) – уравнения многошаговой динамики процесса, получим рекуррентное соотношение, в котором зависимость *от $u(t)$ функции $S(t+1, x(t+1))$* становится *явной*:

$$S(t, x(t)) = \min_{u(t) \subseteq U} [f^0(x(t), u(t)) + S(t+1, f(x(t), u(t)))]$$

$t = 0, 1, \dots, T-1$.

Для оптимального процесса, начинающегося в момент $t = T$, критерий оптимальности (5) сводится к одному последнему слагаемому. Поэтому

$$S(T, x) = F(x). \quad (7)$$

Соотношение (6) и условие (7), играющее роль граничного условия, дают возможность последовательно определять функции $S(x, t)$ из рекуррентного соотношения (6) при $t = T-1, \dots, 1, 0$, а также рассчитать оптимальное управление и оптимальную траекторию, что достигается последовательной реализацией **попятной и прямой процедур** динамического программирования (**прогонка в обратном и прямом времени**).

«Попятная процедура (прогонка в обратном времени)»

При вычислении минимума функции по некоторому аргументу определяются две величины: минимальное значение функции и значение аргумента, при котором минимум достигается. Это значение аргумента может быть неединственным, будем обозначать его символом **arg min**.

Положим $t = T - 1$ в (6) и воспользуемся условием (7). Получим

$$S(T-1, x(T-1)) = \min_{u(T-1) \in U} [f^0(x(T-1), u(T-1)) + F(x(T))] \text{ или с учетом (2)}$$

$$S(T-1, x(T-1)) = \min_{u(T-1) \in U} [f^0(x(T-1), u(T-1)) + F(f(x(T-1), u(T-1)))]$$

Вычисляя этот минимум, найдем функцию $S(T-1, x)$ и значение u , доставляющее данный минимум: $u = u_{T-1}(x)$.

Запись $u_{T-1}(x)$ означает, что значение u зависит не только от номера этапа ($T-1$), но и от состояния x на этом этапе, *от текущего состояния*. Определив $S(T-1, x)$ и полагая $t = T-2$, найдем из (6) функцию $S(T-2, x)$ и соответствующее значение аргумента $u = u_{T-2}(x)$. Продолжая этот процесс в сторону уменьшения t , последовательно получим из (6) функции $S(x, t)$ и $u_t(x)$ при $t = T-1, T-2, \dots, 1, 0$.

Таким образом, попятная процедура состоит в построении функций

$$(8) \quad S(x, t) \text{ и } u_t(x) \text{ для всех } x \text{ и } t = 0, 1, \dots, T.$$

Это построение в отдельных случаях может быть выполнено аналитически, но, как правило, оно является трудоемкой вычислительной процедурой. Вычислительные аспекты обсудим ниже, а пока предположим, что эта процедура, так или иначе реализована.

Прямая процедура (прогонка в прямом времени)

Воспользуемся результатами попятной процедуры для решения исходной задачи, то есть для построения оптимального управления и оптимальной траектории при заданном начальном условии (3).

Полагая $t = 0$ и $x = x_0$ в (8), найдем управление на нулевом шаге: $u(0) = u_0(x_0)$. Далее из соотношения (2) определим на следующем шаге состояние $x(1) = f(x_0, u(0))$. Продолжая этот процесс, найдем $u(1) = u_1(x(1))$, $x(2) = f(x(1), u(1))$ и т.д. Вообще имеем

$$\begin{aligned} u(t) &= u_t(x(t)), \\ x(t+1) &= f(x(t), u(t)), \\ x(0) &= x_0, t = 0, 1, \dots, T-1. \end{aligned} \quad (9)$$

Соотношения (9) определяют **прямую процедуру** и позволяют полностью рассчитать оптимальное управление (**программу управления** для заданных начальных условий) и **оптимальную траекторию**. Минимальное значение критерия оптимальности, отвечающее этой траектории, $J = S(x_0, 0)$.

3. Программное управление и управление с обратной связью (синтез управления)

В теории автоматического регулирования рассматриваются две постановки задачи, связанные с отысканием оптимальных управляющих воздействий: первая задача – определение оптимального **управления в форме программы**, вторая – **в форме синтеза**.

При управлении по заданной программе $u(t)$ решается задача, **как из заданного в момент $t=t_0$ начального состояния x_0 достичь абсциссы t_1 , при наименьшем значении**

критерия качества. В соответствии с терминологией теории автоматического регулирования эта задача связана с управлением по разомкнутому контуру, поскольку информация о реальном текущем состоянии динамической системы $x(t)$ не используется и никак не влияет на формирование управляющего воздействия. Схема управления по разомкнутому контуру показана на рис. 4.

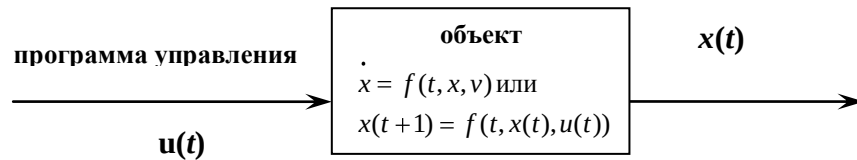


Рис. 4. Схема управления объектом по разомкнутому контуру

В случае синтеза оптимального управления решается более общая задача – **как из любого состояния (t, x) достичь абсциссы t_1 при наименьшем значении функционала качества.** В отличие от программы, синтез оптимального управления оказывается функцией текущего момента времени t и текущего **состояния** $x(t)$ и поэтому позволяет найти закон оптимального управления с обратной связью $u(t, x)$. Синтез $u(t, x)$ реализуется регулятором с идеальной обратной связью.

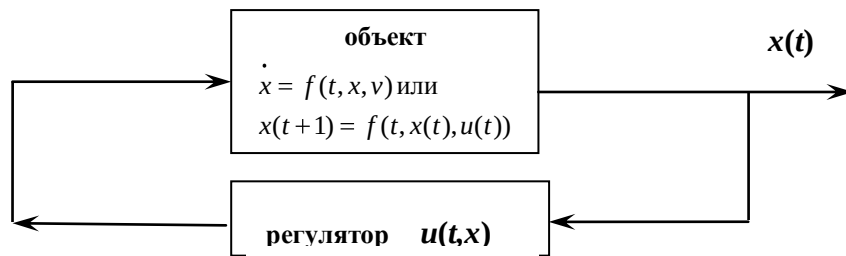


Рис. 5. Схема управления объектом по замкнутому контуру, с обратной связью

В ходе выполнения попятной процедуры метода динамического программирования определяется оптимальное управление **$u(t, x)$** , оно отнесено к моменту t при условии, что система находится в состоянии x . Таким образом, результатом попятной процедуры является синтез управления. Схема управления объектом по замкнутому контуру показана на рис. 5.

Прямая процедура позволяет рассчитать оптимальное управление – программу для любых заданных начальных условий и соответственно оптимальную траекторию. В результате решается **семейство задач оптимального управления** многошаговым процессом, различающихся начальными условиями (t_0, x_0) . **Для всех задач указанного семейства попятные процедуры не различимы, тогда как прямая процедура для каждой задачи семейства индивидуальная и определяется заданными начальными условиями.**

ОБСУЖДЕНИЕ

Попятная и прямая процедуры метода динамического программирования в совокупности дают способ решения поставленной задачи оптимального управления. Одновременно мы получили значительно более общий и универсальный результат. В ходе попятной процедуры построено оптимальное управление по обратной связи (8), то есть управление как функция текущего состояния системы **$u_t(x)=u(t, x)$** . Зная его, нетрудно дать решение задачи оптимального управления при любом начальном условии вида

$x(t) = x^0$, для этого нужно просто реализовать прямую процедуру при этом начальном условии. Подобная реализация не представляет серьезных трудностей и сводится, согласно (9), к вычислению известных функций при конкретных значениях аргументов.

Наибольшую сложность представляет попятная процедура, включающая минимизацию функций по m -мерному векторному аргументу u . Здесь нужно, согласно (6), выполнить T процедур минимизации функций от m переменных. Это, вообще говоря, значительно более простая задача, чем минимизация одной функции по Tm переменным, что требуется при элементарном подходе. Однако есть одно серьезное осложнение. Дело в том, что попятная процедура предусматривает построение функций $S(x, t)$ и $u_t(x)$, зависящих от n -мерного вектора x . По этому вектору x не нужно выполнять операций минимизации, но даже простое табулирование и хранение функций n переменных представляют большие трудности. Если, к примеру, составлять таблицы, придавая каждой переменной 100 значений, то для хранения таблицы одной функции n переменных понадобится $100n$ ячеек памяти. Всего же попятная процедура, в которой участвуют функции $S(x, t)$ и $u_t(x)$, потребует $(m + 1)T! 100n$ ячеек.

Отсюда понятно, что вычислительная реализация метода динамического программирования сталкивается с большими трудностями. Эти трудности Р. Беллман назвал **проклятием размерности**. Для преодоления этих трудностей были предложены подходы [2-5], позволяющие сократить объем вычислений и потребности в памяти для построения оптимального управления. При этом, однако, приходится либо существенно пожертвовать точностью вычислений, либо отказаться от построения управления, оптимального в глобальном смысле, и ограничиться нахождением управлений и траекторий, оптимальных в локальном смысле, то есть по отношению к малым (локальным) вариациям этих траекторий.